

Unit 1 – Oral Answers (Short, Direct)

I. Number Systems

1. $1101_2 = 13$
2. $127_8 = 1010111_2$
3. $2F_{16} = 47$
4. $25_{10} = 11001_2$
5. $10011011_2 = 9B_{16}$
6. $345_{10} = 531_8$
7. $A3_{16} = 10100011_2$
8. $111101_2 = 75_8$
9. $255_{10} = FF_{16}$
10. $1C_{16} = 28$
11. $101101_2 = 55_8$

II. Binary Codes

1. $9 = 1001$ (BCD)
2. $27 = 0010\ 0111$ (BCD)
3. Binary Coded Decimal = **decimal digits in 4-bit form**
4. Used for **decimal accuracy**
5. Binary = full binary; BCD = digit-wise binary
6. $45 = 0100\ 0101$ (BCD)
7. **4 bits** per digit
8. Invalid BCD: **1010–1111**
9. Inefficient due to **extra bits**
10. Advantages: **simple, reliable**

III. Boolean Algebra

1. De Morgan: $(A+B)' = A'B'$, $(AB)' = A' + B'$
2. Dual of $AB + AC = A + BC$

IV. SOP / POS

1. SOP = OR of ANDs; POS = AND of ORs
2. $\Sigma(1,3,5,7)$ = function true at those minterms
3. $\Pi(0,2,4,6)$ = function false at those maxterms
4. Minterm = AND term; Maxterm = OR term

V. K-Maps

1. 3-variable K-map: 8 cells
2. 4-variable K-map: 16 cells

3. Advantage: **quick minimization**
4. Group powers of 2
5. Don't-care = optional cell value
6. As given in expression
7. Minimization = reducing logic
8. Steps: plot → group → simplify
9. Groups: 1,2,4,8
10. Largest group (4-var): **16**
11. Application: **simpler circuits**

VI. Gray / Excess-3

1. $1010_2 \rightarrow \text{Gray} = \mathbf{1111}$
2. $7 \rightarrow \text{Excess-3} = \mathbf{1010}$
3. Purpose: one-bit change
4. Gray = single-bit transitions
5. $1101 \text{ (Gray)} \rightarrow \mathbf{1011}$ (binary)
6. Used in encoders
7. Add 3
8. Reflected binary = Gray code
9. BCD $\rightarrow$ Excess-3: add 0011

VII. Binary Arithmetic

1. $1011 + 1101 = \mathbf{11000}$
2. $1111 + 0001 = \mathbf{10000}$
3. $100 + 110 = \mathbf{1010}$
4. Carry moves to next bit
5. Basic rules: $0+0=0$, $0+1=1$, $1+1=10$
6. $1101 - 1010 = \mathbf{0011}$
7. $1000 - 0011 = \mathbf{0105}$ (binary **0101**)
8. $1001 - 0110 = \mathbf{0011}$
9. Borrow rules: borrow from next bit
10. 1's comp of $1101 = \mathbf{0010}$
11. 1's comp of $00001111 = \mathbf{11110000}$
12. $-9 \rightarrow \mathbf{0110}$ (4-bit 1's comp)
13. $1011 - 0110 = \mathbf{0101}$ (1's comp)
14. End-around carry added back

Unit 2 – Oral Answers (Short, Direct)

I. Multiplexers (MUX)

2:1 Multiplexer

1. 2:1 MUX = selects **one of two inputs**.
2. Block diagram = **2 inputs** → **1 output via selector**.
3. Truth table: $Y = S' \cdot I_0 + S \cdot I_1$.
4. Applications: **data routing, signal selection**.

4:1 Multiplexer

1. 4:1 MUX = **4 inputs** → **1 output**.
2. Needs **2 select lines**.
3. Internal diagram: uses **gates and selectors**.
4. Truth table: Y = based on **S1S0 selection**.
5. Applications: **function generation, communication**.

8:1 Multiplexer

1. 8:1 MUX = **8 inputs**.
2. Needs **3 select lines**.
3. Logic diagram = **AND/OR gating**.
4. Used in **ALUs, data routing**.

16:1 Multiplexer

1. 16:1 MUX = **16 inputs** → **1 output**.
2. Needs **4 select lines**.
3. Working: output based on **4-bit selector**.
4. Used in **large data path systems**.

Cascading Multiplexers

1. 16:1 using 4×4:1: **cascade 4 MUX** → **final 4:1**.
2. Cascading = **connecting MUXes in stages**.

Types & Advantages of MUX

1. Types: 2:1, 4:1, 8:1, 16:1.
 2. Advantages: **reduces wiring, implements logic easily**.
-

II. Adders & Subtractors

Half Adder

1. Half adder = **adds 2 bits**.
2. Truth table: Sum = XOR, Carry = AND.
3. Limitation: **no carry-in**.

Full Adder

1. Full adder = **adds 3 bits**.
2. Difference: has **carry-in**.
3. Truth table: standard **S and Cout**.
4. Built using **two half adders + OR**.

Half Subtractor

1. Half subtractor = subtracts **A - B**.
2. Logic diagram: uses **XOR & AND**.
3. Truth table: **Diff = $A \oplus B$** , Borrow = **$A'B$** .

Full Subtractor

1. Full subtractor = subtracts with **borrow-in**.
 2. Diagram uses **two half subtractors + OR**.
 3. Truth table: standard DIFF/BO.
 4. **$DIFF = A \oplus B \oplus Bin$** , Borrow = **$A'B + (A \oplus B)'Bin$** .
 5. Built using **two half subtractors + OR**.
-

III. Decoders

1. Applications: **memory selection, instruction decoding**.
 2. Decoder vs Demux: Decoder = **no data input**, DEMUX = **routes data**.
-

IV. Parity Circuits

1. Parity bit = **error detecting bit**.
 2. Even parity: total 1s even. Odd parity: total 1s odd.
 3. Parity checker detects error if **mismatch occurs**.
 4. Used in **communication, memory systems**.
-

V. Difference-Based Questions

1. MUX vs DEMUX: **many→one vs one→many**.
2. Decoder vs DEMUX: Decoder has **no data input**.
3. MUX vs Decoder: MUX = **selects**, Decoder = **identifies code**.
4. DEMUX preferred for **data distribution**.

Unit 3 – Oral Answers (Short, Direct)

I. Basics of Flip-Flops

1. Flip-flop = **1-bit storage element**.
 2. Called 1-bit memory because it **stores 0 or 1**.
 3. Combinational = no memory; Sequential = **has memory**.
 4. Clock = **controls timing** of state changes.
-

II. S-R Flip-Flop

1. S-R latch = **two cross-coupled NOR/NAND gates**.
 2. Truth table: $S=1 \Rightarrow \text{Set}$; $R=1 \Rightarrow \text{Reset}$.
 3. $S=1$, $R=1$ is **invalid**.
 4. Latch = **level-triggered**; Flip-flop = **edge-triggered**.
-

III. J-K Flip-Flop

1. J-K diagram = **feedback-based FF**.
 2. Modes: **Set, Reset, Toggle, No-change**.
 3. Toggle = switch state; Reset = 0; Set = 1; No-change = hold.
 4. Race-around = **continuous toggling**.
 5. Happens in **level-triggered J-K FFs**.
 6. Master-slave FF **removes race** by using two stages.
-

IV. D Flip-Flop

1. D FF: **Q follows D** on clock.
 2. Truth table: $Q = D$.
 3. Excitation table: To get next state, set **$D = Q(\text{next})$** .
 4. Called delay FF because output is **delayed copy of input**.
-

V. T Flip-Flop

1. T FF: $T=1 \Rightarrow \text{toggle}$; $T=0 \Rightarrow \text{hold}$.
 2. Truth table: $Q(\text{next}) = Q \oplus T$.
 3. Used in **counters**.
-

VI. Characteristics of Flip-Flops

1. Characteristics: **setup time, hold time, propagation delay.**
 2. Propagation delay = time for **output to update.**
-

VII. Registers

1. Register = group of flip-flops; stores **multiple bits.**
 2. Bidirectional shift register = shifts **left and right.**
 3. Ring counter & Johnson counter = **shift-register counters.**
 4. Applications: **data storage, serial-to-parallel conversion.**
-

VIII. Counters – Asynchronous (Ripple)

1. Asynchronous counter = FFs **triggered one after another.**
 2. Called ripple because **clock ripples** through FFs.
 3. 4-bit UP counter: **0000 → 1111** sequence.
-

IX. Counters – Synchronous

1. Synchronous counter = **all FFs clocked together.**
 2. Faster because **no ripple delay.**
 3. Synchronous counters are **faster and more reliable.**
-

X. Excitation Tables & Design

1. Excitation table = **required inputs for transitions.**
 2. Important for **counter/state machine design.**
 3. Used to **derive flip-flop inputs.**
-

XI. Standard IC Counters

1. IC 7490 = **decade counter.**
2. Applications: **frequency division, clocks.**
3. IC 7493 = **4-bit binary counter.**
4. Applications: **timers, dividers.**

Unit 4 – Oral Answers (Short, Direct)

I. Architecture & Organization (8086)

1. BIU = **Fetches instructions**, handles addressing.
 2. EU = **Executes instructions**, ALU operations.
 3. BIU fetches while EU executes $\Rightarrow$ **parallel speed-up**.
 4. Physical address = **(Segment $\times$ 16) + Offset**.
 5. Segmentation = memory divided into **64 KB segments**.
-

II. Registers in 8086

1. Registers: **General-purpose, Segment, Pointer/Index, Flags**.
 2. GP registers (AX, BX, CX, DX) used for **data/arithmetics**.
 3. Segment registers define **memory segments**.
 4. Segment $\neq$ GP: used only for **address formation**.
 5. AX=AH+AL, BX=BH+BL, CX=CH+CL, DX=DH+DL.
 6. Examples: AL, AH, BL, BH etc. (8-bit parts).
 7. Flag register = **condition & control flags**.
 8. Status flags show result; control flags **affect operations**.
-

III. Addressing Modes

A. Immediate, Direct, Register

1. Immediate = value inside instruction.
2. Direct = **memory address given directly**.
3. Register = operand in **register**.

B. Register Indirect, Indexed, Relative

1. Register indirect = address in **BX, BP, SI, DI**.
2. Indexed = base address + **SI/DI**.
3. Relative = PC + **offset**.

C. Base, Based Indexed, String

1. Base = address using **BX/BP**.
2. Based indexed = base + **index**.
3. String addressing uses **SI, DI** automatically.

D. I/O Addressing

1. Direct I/O = port number given.
 2. Indirect I/O = port in **DX**.
 3. Implied = operand assumed (e.g., **CLC**, **STC**).
-

IV. Instruction Set

1. Types: **Data transfer, Arithmetic, Logical, Branch, String, I/O**.
2. Format: **Opcode + operands**.

A. Data Transfer

1. MOV = copy data.
2. XCHG = swap; LEA = load address; PUSH/POP = stack ops.

B. Arithmetic

1. ADD/SUB, MUL/DIV = basic arithmetic.
2. INC/DEC = +1 or -1.

C. Branch

1. JMP = unconditional jump; CALL = subroutine call.
2. Conditional jumps depend on **flags**.

D. Logical

1. AND, OR, XOR = bit ops.
2. NOT = invert; TEST = AND w/o storing.

E. Bit Manipulation

1. SHL/SHR = shifts; ROL/ROR = rotates.
-

V. Bus System & Modes

A. Multiplexed Bus

1. AD0-AD15 = **address/data shared lines**.

B. Minimum vs Maximum Mode

1. Min mode = **single processor**; Max mode = **multiprocessor**.
2. Min mode signals: **ALE, RD, WR, DEN, DT/R**.
3. Max mode signals: **RQ/GT, LOCK, S0-S2**.

VI. Pin Descriptions

Group 1: Basic Buses

1. AD0-AD15 = addr/data; BHE' = high-byte enable; READY = wait; RD' = read.

Group 2: Interrupt & Control Pins

1. RESET = restart; INTR = interrupt request; NMI = non-maskable interrupt; TEST = wait for external test.

Group 3: Power & Clock

1. CLK = clock; VCC = power; INTA' = interrupt acknowledge; ALE = address latch enable.

Group 4: Bus Control & DMA

1. LOCK' = bus lock; RQ'/GT' = DMA request/grant; HOLD = bus request; HLDA = bus grant acknowledge.

Group 5: Read/Write Mode

1. WR' = write; MN/MX' = mode select; DEN' = enable data bus; DT/R' = data transmit/receive.

Unit 5 – Oral Answers (Short, Direct)

A. Introduction to Pentium

1. Pentium = **important Intel processor** with **superscalar architecture**. Considered a milestone due to **dual pipelines, higher speed, 64-bit data bus**.
-

B. CPU Pin Description

1. Address pins = **select memory locations**; Data pins = **transfer data**; Control pins = **manage read/write/interrupts**.
 2. During interrupts, interrupt pins **signal request, acknowledge, and service routines**.
 3. System pins = **power, clock, reset**; Control pins = **bus control, read/write signals**.
-

C. RISC Concepts in Pentium

1. RISC = **simple fast instructions**. Pentium not pure RISC because it has **CISC-style complex instructions**.
 2. Pentium mixes RISC (pipelines, fast execution) + CISC (variable-size instructions, many addressing modes).
-

D. Bus Operations

1. Bus operations = **read/write cycles and signal transfers**.
 2. Bus cycles: **memory read, memory write, I/O read, I/O write, interrupt cycles**.
-

E. Superscalar Architecture

1. Superscalar = **executes multiple instructions per cycle**.
 2. Pentium U/V pipelines run **in parallel**, unlike single scalar pipeline.
-

F. Pipelining

1. Instruction pipelining = **overlapping stages** of execution.
 2. Main stages: **Fetch → Decode → Execute → Write-back**.
-

G. Branch Prediction

1. Branch prediction = **guess next path** to avoid stalls; essential for pipeline efficiency.
-

H. Instruction & Data Caches

1. Instruction cache stores **opcodes**; Data cache stores **operands** → increases speed.

Unit 6 – Oral Answers (Short, Direct)

A. Interrupt Structure

1. Interrupt = **signal that stops normal execution**. Types: **Hardware, Software, Maskable, Non-maskable**.
 2. 8086 interrupt structure = **vector table + priority system + ISR execution flow**.
-

B. ISR (Interrupt Service Routine) & Vector Table

1. ISR = **routine executed when an interrupt occurs**.
 2. 8086 ISR sequence: **Push flags → Push CS/IP → Load new CS:IP → Execute ISR → IRET**.
-

C. Hardware & Software Interrupts

1. Hardware interrupt = from external device (e.g., **INTR, NMI**). Software interrupt = via **INT instruction**.
-

D. INTR and NMI Interrupt Response

1. INTR = **maskable interrupt**; NMI = **non-maskable interrupt**.
 2. Maskable interrupts can be **disabled**; NMI **cannot be disabled** (used in safety-critical systems).
-

E. Execution of ISR & Priority

1. 8086 ISR execution steps: **Acknowledge → Save context → Execute ISR → Restore → Return**.
-

F. Programmable Interrupt Controller (8259)

1. ICW1–4 = **initialize PIC** (mode, vector base, cascading, control settings).
2. OCW1–3 = **runtime control** (masking, priority, end-of-interrupt, polling).